

GROCAL

(Growth + Circle)

Production-Ready Flutter Frontend

Architecture Specification

Prepared for Cursor AI / Senior Flutter Developers
Complete Clean Architecture + Laravel Backend Ready

This document contains the complete technical specification for building the full production-grade Flutter frontend for the GroCal platform. Every requirement, algorithm, and architectural decision is documented.

ROLE & MISSION

You are a **Senior Flutter Architect and Product Engineer**.

Project Name: GroCal (Growth + Circle)

Mission:

Transform the existing GroCal frontend from a demo/mock-data application into a complete **production-ready frontend architecture** connected through proper API service layers for a future Laravel backend.

IMPORTANT RULES

1. Remove **ALL** hardcoded, demo, mock, or fake data from the entire application.
2. Every screen must consume data **exclusively** through repositories and API service layers.
3. **Never** call APIs directly inside UI widgets.
4. Implement **Clean Architecture** with clear separation of concerns (Data, Domain, Presentation).
5. Prepare every feature for seamless integration with **Laravel REST APIs**.
6. Create models, repositories, service classes, state management (Riverpod / Bloc / Cubit), DTOs, and strict API contracts.
7. All UI must continue working even before the backend exists by using API abstraction + repository pattern (mock repositories during development are allowed only if clearly separated).
8. Use **scalable enterprise architecture** suitable for production apps.

TECH STACK (Mandatory)

- Flutter (latest stable)
- Dio (for HTTP client)
- Flutter Secure Storage (tokens & sensitive data)
- WebSocket support (for real-time chat)
- Repository Pattern + Clean Architecture
- Pagination + Caching strategy
- Token-based authentication (Access + Refresh tokens)
- Laravel API Ready (follow REST conventions)

AUTHENTICATION

Features Required:

- Login
- Register (with welcome bonus logic)
- Forgot Password
- Email Verification
- Refresh Token
- Logout
- Remember Me

Required Components:

- AuthService
- AuthRepository

- **Models:** User, Role, Permission, Token

PROFILE SYSTEM

Each user profile must contain:

Basic Info:

- Profile Picture
- Cover Image
- Full Name
- Username (unique)
- Bio
- Location
- Website

Social Accounts:

- YouTube
- Instagram
- TikTok
- Facebook
- LinkedIn
- X (Twitter)

Statistics:

- Followers
- Following
- Completed Tasks
- Credits
- Level
- Reputation Score

Actions:

- Edit Profile
- Follow / Unfollow User
- Block User
- Report User

CREDIT ECONOMY SYSTEM

- Every user has a credit wallet.
- Credits can be **Earned**, **Spent**, or **Transferred**.

Credit Wallet must track:

- Current Credits
- Lifetime Credits Earned
- Lifetime Credits Spent
- Full **Transaction History** (Date, Amount, Reason, Reference ID).
- Credits used in tasks must go into **Escrow** until task is approved.

API-ready architecture required.

CREATOR LEVEL SYSTEM

Levels:

- Beginner
- Bronze
- Silver
- Gold
- Platinum

Level is calculated from:

- Total Credits Earned
- Completed Tasks
- Account Age
- Reputation Score

Level badges must be displayed across the app (profile, task creation, etc.).

TASK MARKETPLACE SYSTEM (Core Feature)

Task Types:

- Watch Video
- Like Content
- Follow Account
- Subscribe Channel
- Comment Content

Supported Platforms:

- YouTube
- Instagram
- TikTok
- Facebook

CREATE TASK FLOW (Complete Algorithm)

Step 1: User inputs

- Task Title
- Task Description
- Task Type
- Platform
- Target Link (content URL)
- Reward Per Participant
- Desired Participant Count

Step 2: System validates

- Valid URL format for selected platform
- Supported platform
- Duplicate task detection
- User reputation meets minimum requirement

Step 3: System calculates

- Total Credits Required = Reward Per Participant × Participant Count
- Platform Fee = 5% to 10% (configurable)

- Final Cost = Total Credits Required + Platform Fee

Example:

- Reward: 5 credits | Participants: 100
- Task Cost: 500 credits | Platform Fee: 50 credits

Total Deduction: 550 credits

Step 4: Wallet Check

- If sufficient credits → Create task + move amount to **Escrow Wallet**
- Else → Show clear error: **"Insufficient Credits"**

Step 5: Credits are **locked** in Escrow (not transferred immediately).

TASK STATUS FLOW

Draft → Pending Review → Active → In Progress → Completed → Closed

TASK VERIFICATION SYSTEM

1. Creator provides content URL.
2. Participant opens task → redirected to platform.
3. After completing action, participant pastes **Verification URL**.
4. System verifies:
 - Platform Match
 - Content ID Match (exact match required)
 - Watch Time Protection (minimum 60% of video length)
 - Duplicate check (user has not completed this task before)
 - Device validation

Prepare frontend for future AI verification (Trust Score system).

Task Statuses: Pending, Under Review, Approved, Rejected.

TASK MANAGEMENT SCREENS

- My Tasks
- Created Tasks
- Completed Tasks
- Pending Tasks
- Rejected Tasks

Task Analytics (for creators):

- Participants Joined
- Participants Completed
- Credits Spent

COMMUNITY SYSTEM

Community Types:

- Public
- Private
- Invite Only

Community Settings:

- Name
- Description
- Avatar
- Banner
- Rules
- Tags

Member Roles:

- Owner
- Admin
- Moderator
- Member

Permissions (Role-based):

- Promote / Demote Member
- Mute / Ban Member
- Approve / Reject Join Requests

JOIN REQUEST SYSTEM (Private Communities)

- Users can request to join.
- Admins can Approve or Reject.
- Dedicated **Pending Requests** screen required.

REAL-TIME GROUP CHAT (WhatsApp-style)

Features:

- Real-time messaging
- Reply, Edit, Delete messages
- Pinned messages
- Message reactions
- Media messages
- Voice notes (placeholder)
- File sharing (placeholder)

Prepare for Laravel WebSockets / Reverb.

COMMUNITY INFO PAGE

Display: Name, Description, Creation Date, Member Count, Admins list, Moderators list.

Settings Screen: Group Privacy, Invite Permissions, Member Permissions, Content Permissions.

PLATFORM ADMIN ANNOUNCEMENT COMMUNITY

(Special System Community)

- Created automatically.
- All users are automatically joined.
- **Only Platform Admins** can post.
- Regular users can: Read, React, Comment.
- Users **cannot** Post, Delete, or Edit.

Protected frontend architecture required.

NOTIFICATION SYSTEM

Notification Types:

- Likes, Comments, Follows
- Task Completed / Approved / Rejected
- Community Join Requests
- Admin Announcements

Real-time ready architecture required.

SEARCH SYSTEM

Global Search supporting:

- Users
- Communities
- Tasks

Pagination required.

REPORTING SYSTEM

Report any of the following with a **required reason**:

- User
- Task
- Community
- Message

SETTINGS

- Account Settings
- Privacy Settings
- Notification Settings
- Community Settings
- Task Settings

SECURITY & ROLE-BASED ACCESS CONTROL (RBAC)

Roles:

- User
- Moderator
- Admin
- Super Admin

Requirements:

- Permission guards on UI and business logic.
- Hide restricted UI elements automatically based on role.

API STRUCTURE (Mandatory)

Create **service classes only** (no direct API calls in UI):

- AuthService
- UserService
- TaskService
- CommunityService
- ChatService
- NotificationService
- CreditService

Rules:

- Use **Dio**.
- No hardcoded URLs anywhere.
- Use environment configuration (.env or flavor-based).
- Support all HTTP methods: GET, POST, PUT, PATCH, DELETE.

STATE MANAGEMENT

- Refactor current frontend to **centralized state management**.
- No business logic or API state kept inside individual screens.
- Support states: **Loading, Success, Error, Empty**.

GROCAL TASK MARKETPLACE ALGORITHMS (Critical Business Logic)

1. User Registration

When a new user creates an account:

- System automatically creates a wallet.
- **Welcome Bonus = 500 Credits**.
- User Level = **Beginner**.
- Reputation Score = **100**.

2. Task Creation Algorithm (Full Flow)

Already detailed above in "CREATE TASK FLOW".

3. Participant Flow

- User opens marketplace.
- System filters out:
 - Own tasks
 - Already completed tasks
 - Tasks from blocked users
 - Reported tasks

4. Task Execution & Watch Time Protection

- User clicks **Start Task** → TaskSession is created (Session ID, Task ID, User ID, Start Time, Status, Device ID).
- User is redirected to the platform.
- Minimum watch time = **60%** of content length.

- If user returns too quickly → **Reject** with reason "Minimum watch time not met".

5. Link Verification Algorithm

After completion, user submits Verification URL. System checks (in order):

1. **Platform Match**
2. **Content Match** (exact Content ID comparison)
3. **Duplicate Detection** (has user already completed this task?)
4. **Device Validation** (flag if same device is completing excessive tasks)

6. Automatic Approval

Auto-approve **only if ALL** are true:

- Platform Match = TRUE
- Content Match = TRUE
- Watch Time = PASS
- Duplicate = FALSE
- Fraud Score = LOW

7. Credit Distribution (On Approval)

- Escrow Wallet → Participant Wallet (+ Reward)
- Creator Escrow → Deducted

8. Reputation System

- Approved task → **+2** Reputation
- Rejected task → **-5** Reputation
- Reported for fraud → **-20** Reputation

9. Level System (Exact Thresholds)

- **Beginner**: 0 – 999 Credits
- **Bronze**: 1,000 – 4,999 Credits
- **Silver**: 5,000 – 14,999 Credits
- **Gold**: 15,000 – 49,999 Credits
- **Platinum**: 50,000+ Credits

FRAUD DETECTION & PROTECTION

Detect:

- Multiple accounts from same device
- Duplicate devices
- VPN abuse
- Same link spam
- Repeated fake verifications
- Automated task completion

Task Creator Protection:

- If creator deletes content → Task automatically paused + remaining credits refunded.

Manual Review System:

- High fraud score → Status = **Under Review**
- Moderator can: Approve, Reject, or Ban User.

Future AI Verification (Prepare Frontend):

AI will analyze:

- Watch Duration
- Real Engagement
- Comment Quality
- User Behavior
- Fraud Probability

AI assigns **Trust Score (0-100)**:

- Above threshold → **Auto Approved**
- Below threshold → **Manual Review**

DELIVERABLES

Refactor the **entire existing GroCal frontend** into a production-grade codebase with:

- Clean Architecture (Data / Domain / Presentation layers)
- Repository + Service layer for **every** feature
- Proper Models + DTOs
- State management with Loading/Success/Error/Empty states
- Route guards + Role-based UI protection
- Complete Task Marketplace with all algorithms above
- Real-time chat architecture (ready for WebSockets)
- Community system with roles & permissions
- Credit wallet + escrow system
- Notification system (real-time ready)
- Global search with pagination
- Reporting system
- Full settings & security screens

Final codebase must be ready for Laravel backend integration with zero major rewrites needed on the frontend side.

Recommended Folder Structure (Follow this):

```
lib/
├── core/
│   ├── constants/
│   ├── errors/
│   ├── network/           (Dio setup + interceptors)
│   ├── utils/
│   └── config/
├── features/
│   ├── auth/
│   ├── profile/
│   ├── task/              (marketplace + verification)
│   ├── community/        (includes chat)
│   ├── wallet/
│   ├── notifications/
│   └── search/
├── shared/
│   ├── models/
│   ├── repositories/
│   ├── services/
│   └── widgets/
```

■■■ app/ (routes, theme, main.dart)

This specification is complete and ready for implementation. Every requirement, business rule, and architectural decision has been documented with no omissions.